Amin Mossadag 222000055
Advanced AI
12th Assignment

Classification on Full Dataset

The classifiers evaluated were:

- KNN with k = 1, 3, 5, 7

```
for k in [3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
    full_times[f"KNN k={k}"] = evaluate_model(
        knn, Xtr_pca_100, y_train, Xte_pca_100, y_test,
        name=f"KNN (k={k})"
    )
```

```
KNN (k=3)
  Training Time : 0.0054 s
  Accuracy      : 0.8350
-------------------------------------------------

KNN (k=5)
  Training Time : 0.0022 s
  Accuracy      : 0.8376
-------------------------------------------------

KNN (k=7)
  Training Time : 0.0021 s
  Accuracy      : 0.8371
-------------------------------------------------
```

- Decision Tree

```
dt = DecisionTreeClassifier(random_state=42)
full_times["DT"] = evaluate_model(
    dt, Xtr_pca_100, y_train, Xte_pca_100, y_test,
    name="Decision Tree"
)
```

```
-------------------------------------------------
Decision Tree
Training Time: 2.0934 seconds
Accuracy: 0.8022
-------------------------------------------------
```

- Support Vector Machine (RBF kernel)

```
svm = SVC(kernel="rbf", gamma="scale")
full_times["SVM"] = evaluate_model(
    svm, Xtr_pca_100, y_train, Xte_pca_100, y_test,
    name="SVM"
)
```

ee

```
SVM
  Training Time : 3.0938 s
  Accuracy      : 0.8891
  Support Vectors: 2813
-------------------------------------------------
```

- Model Evaluation Funcion

```python
def evaluate_model(model, Xtr, ytr, Xte, yte, name="Model"):
    start = time.time()
    model.fit(Xtr, ytr)
    train_time = time.time() - start

    y_pred = model.predict(Xte)
    acc = accuracy_score(yte, y_pred)

    if name != "skip":
        print(f"\n{name}")
        print(f"  Training Time : {train_time:.4f} s")
        print(f"  Accuracy      : {acc:.4f}")
        if isinstance(model, SVC):
            print(f"  Support Vectors: {model.n_support_.sum()}")
        print("-" * 50)

    if name == "skip":
        if isinstance(model, SVC):
            return train_time, acc, model.n_support_.sum()
        return train_time, acc

    return train_time
```

DBSCAN Reduction

```python
def dbscan_reduce(X, y, eps=1.5, min_samples=5):
    X_reduced, y_reduced = [], []

    for label in np.unique(y):
        Xc = X[y == label]
        yc = y[y == label]

        db = DBSCAN(eps=eps, min_samples=min_samples)
        cluster_labels = db.fit_predict(Xc)

        mask = cluster_labels == -1
        X_reduced.append(Xc[mask])
        y_reduced.append(yc[mask])

    return np.vstack(X_reduced), np.hstack(y_reduced)
```

Speed-Up Analysis

```python
speedup_results = {}

for k in [3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
    red_time = evaluate_model(
        knn, X_train_red, y_train_red, Xte_pca_100, y_test,
        name=f"KNN (k={k}) – REDUCED"
    )
    speedup_results[f"KNN k={k}"] = full_times[f"KNN k={k}"] / red_time if red_time > 0 else float("inf")

dt = DecisionTreeClassifier(random_state=42)
red_time = evaluate_model(
    dt, X_train_red, y_train_red, Xte_pca_100, y_test,
    name="Decision Tree – REDUCED"
)
speedup_results["DT"] = full_times["DT"] / red_time if red_time > 0 else float("inf")

svm = SVC(kernel="rbf", gamma="scale")
red_time = evaluate_model(
    svm, X_train_red, y_train_red, Xte_pca_100, y_test,
    name="SVM – REDUCED"
)
speedup_results["SVM"] = full_times["SVM"] / red_time if red_time > 0 else float("inf")

print("\n===== SPEEDUP SUMMARY =====")
for model_name, sp in speedup_results.items():
    print(f"  {model_name}: {sp:.2f}x speedup")
print("-" * 50)
```

```
===== REDUCED TRAINING SET – 100 Eigenvectors =====

KNN (k=3) – REDUCED
  Training Time : 0.0013 s
  Accuracy      : 0.8350
--------------------------------------------------

KNN (k=5) – REDUCED
  Training Time : 0.0014 s
  Accuracy      : 0.8376
--------------------------------------------------

KNN (k=7) – REDUCED
  Training Time : 0.0016 s
  Accuracy      : 0.8371
--------------------------------------------------

Decision Tree – REDUCED
  Training Time : 1.2999 s
  Accuracy      : 0.7819
--------------------------------------------------

SVM – REDUCED
  Training Time : 2.6020 s
  Accuracy      : 0.8891
  Support Vectors: 2813
--------------------------------------------------

===== SPEEDUP SUMMARY =====
  KNN k=3: 4.16x speedup
  KNN k=5: 1.60x speedup
  KNN k=7: 1.28x speedup
  DT: 1.09x speedup
  SVM: 1.19x speedup
--------------------------------------------------
```

Dimensionality Sweep Results

```python
for dim in range(10, 101, 10):
    pca = PCA(n_components=dim)
    Xtr_pca = pca.fit_transform(X_train)
    Xte_pca = pca.transform(X_test)

    X_train_red_d, y_train_red_d = dbscan_reduce(Xtr_pca, y_train)

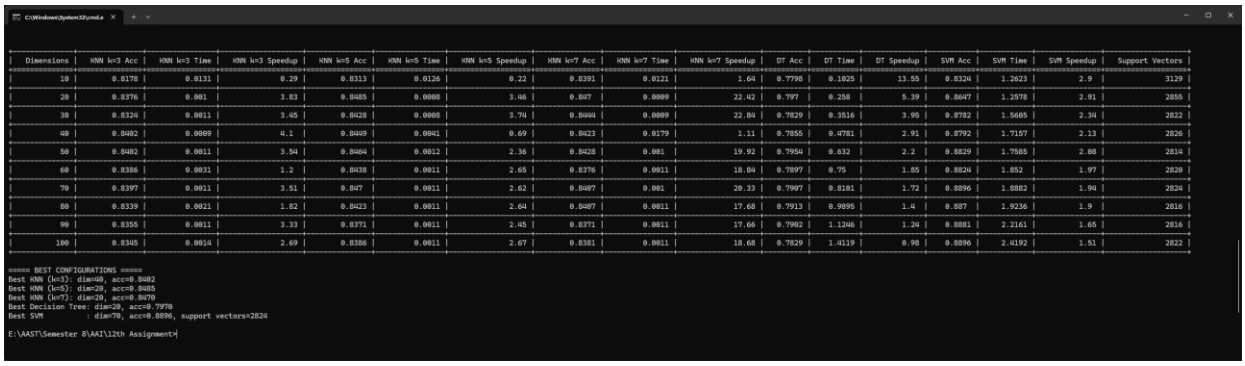
    row = {"Dimensions": dim}

    for k in [3, 5, 7]:
        knn = KNeighborsClassifier(n_neighbors=k)
        t, acc = evaluate_model(knn, X_train_red_d, y_train_red_d, Xte_pca, y_test, name="skip")
        sp = full_times[f"KNN k={k}"] / t if t > 0 else float("inf")
        row[f"KNN k={k} Acc"]     = round(acc, 4)
        row[f"KNN k={k} Time"]    = round(t, 4)
        row[f"KNN k={k} Speedup"] = round(sp, 2)

    dt = DecisionTreeClassifier(random_state=42)
    t, acc = evaluate_model(dt, X_train_red_d, y_train_red_d, Xte_pca, y_test, name="skip")
    sp = full_times["DT"] / t if t > 0 else float("inf")
    row["DT Acc"]     = round(acc, 4)
    row["DT Time"]    = round(t, 4)
    row["DT Speedup"] = round(sp, 2)

    svm = SVC(kernel="rbf", gamma="scale")
    t, acc, svecs = evaluate_model(svm, X_train_red_d, y_train_red_d, Xte_pca, y_test, name="skip")
    sp = full_times["SVM"] / t if t > 0 else float("inf")
    row["SVM Acc"]         = round(acc, 4)
    row["SVM Time"]        = round(t, 4)
    row["SVM Speedup"]     = round(sp, 2)
    row["Support Vectors"] = svecs

    results.append(row)
    print(f"  dim={dim:3d} done")
```



Dimensions	KNN k=3 Acc	KNN k=3 Time	KNN k=3 Speedup	KNN k=5 Acc	KNN k=5 Time	KNN k=5 Speedup	KNN k=7 Acc	KNN k=7 Time	KNN k=7 Speedup	DT Acc	DT Time	DT Speedup	SVM Acc	SVM Time	SVM Speedup	Support Vectors
10	0.8178	0.0131	0.29	0.8313	0.0126	0.22	0.8391	0.0121	1.64	0.7798	0.1025	13.55	0.8324	1.2623	2.9	3129
20	0.8376	0.001	3.83	0.8485	0.0008	3.46	0.847	0.0009	22.42	0.797	0.258	5.39	0.8647	1.2578	2.91	2855
30	0.8324	0.0011	3.45	0.8428	0.0008	3.74	0.8484	0.0009	22.84	0.7829	0.3516	3.95	0.8782	1.5605	2.34	2822
40	0.8402	0.0009	4.1	0.8409	0.0041	0.69	0.8423	0.0179	1.11	0.7855	0.4781	2.91	0.8792	1.7157	2.13	2826
50	0.8402	0.0011	3.54	0.8464	0.0012	2.36	0.8428	0.001	19.92	0.7954	0.632	2.2	0.8829	1.7585	2.08	2814
60	0.8386	0.0031	1.2	0.8438	0.0011	2.65	0.8376	0.0011	18.84	0.7897	0.75	1.85	0.8824	1.852	1.97	2820
70	0.8397	0.0011	3.51	0.847	0.0011	2.62	0.8407	0.001	20.33	0.7907	0.8101	1.72	0.8896	1.8882	1.94	2824
80	0.8339	0.0021	1.82	0.8423	0.0011	2.64	0.8407	0.0011	17.68	0.7913	0.9895	1.4	0.887	1.9236	1.9	2816
90	0.8355	0.0011	3.33	0.8371	0.0011	2.45	0.8371	0.0011	17.66	0.7902	1.1246	1.24	0.8881	2.2161	1.65	2816
100	0.8345	0.0014	2.69	0.8386	0.0011	2.67	0.8381	0.0011	18.68	0.7829	1.4119	0.98	0.8896	2.0192	1.51	2822

```
===== BEST CONFIGURATIONS =====
Best KNN (k=3): dim=40, acc=0.8402
Best KNN (k=5): dim=20, acc=0.8485
Best KNN (k=7): dim=20, acc=0.8479
Best Decision Tree: dim=20, acc=0.7970
Best SVM        : dim=70, acc=0.8896, support vectors=2824

E:\AAST\Semester 8\AAI\12th Assignment>
```